

Containers and virtual machines provide strong workload isolation and remain indispensable components of modern infrastructure. However, they primarily focus on isolating processes and resources. AI agents introduce additional requirements. Infrastructure and development teams must control external connectivity, protect secrets from agent visibility, govern model access, and apply runtime policies to actions influenced by prompts and external inputs.

OpenShell addresses these concerns through runtime controls while relying on existing Linux isolation mechanisms underneath.

OpenShell architecture

OpenShell separates control-plane responsibilities from local runtime enforcement.

The ‘gateway’ acts as the control plane and is responsible for authentication, policy management, and sandbox lifecycle operations.

Agents execute inside isolated ‘sandbox’ environments. Within the sandbox, the ‘supervisor’ acts as the local enforcement point.

The ‘policy engine’ evaluates policies governing filesystem access, process execution, network communication, and inference behaviour.

The ‘privacy router’ helps isolate credentials and manages model interactions without exposing sensitive information directly to the agent environment.

The AI workload itself runs as a restricted process under the supervision of the local enforcement layer. Filesystem and process boundaries are established when the sandbox is created, while communication and inference policies remain logically separated from those boundaries.

Core protection layers

Filesystem layer: OpenShell uses Landlock to restrict filesystem visibility. Agents receive access only to explicitly allowed paths, with enforcement performed by the Linux kernel. Since these restrictions are enforced by the kernel, they remain effective regardless of application behaviour.

Process layer: OpenShell relies on standard Linux mechanisms such as seccomp (secure computing mode) filters and privilege reduction to minimise the attack surface and help prevent privilege escalation.

Network layer: Network policies determine which destinations agents are permitted to contact. OpenShell separates communication policies from filesystem boundaries established during sandbox creation, allowing network access to be governed independently.

Inference layer: Instead of exposing provider credentials directly to the agent process, inference requests are routed through privacy-aware interfaces. Credentials remain outside the agent environment, reducing the

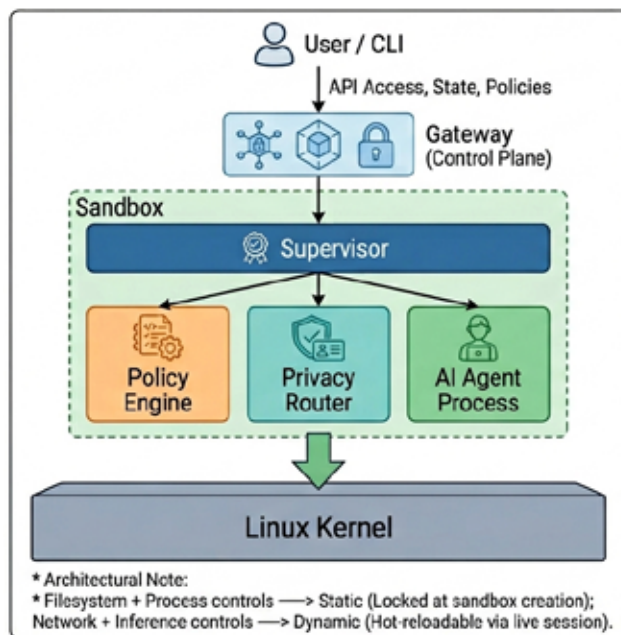


Figure 1: OpenShell architecture

possibility of accidental disclosure.

OpenShell and Linux kernel primitives

OpenShell does not introduce a new kernel isolation mechanism. Instead, it acts as a user space framework built on established Linux security primitives.

The supervisor establishes security boundaries and launches the agent process inside those constraints.

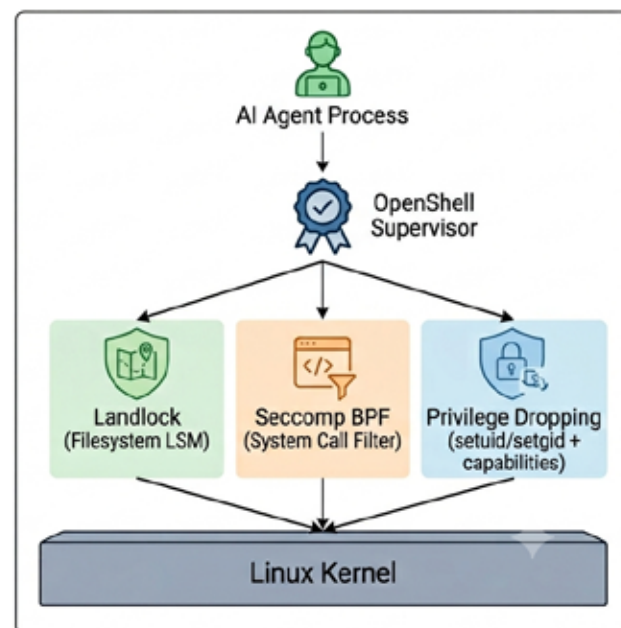


Figure 2: Linux kernel primitives used by OpenShell